Ders 21 Genişletilmiş Notu: Özyinelemeli Algoritmaların Analizi: Recursion Tree, Merge Sort, Recurrence Bağıntıları ve Lab-3

Ders 21 Genişletilmiş Notu: Özyinelemeli Algoritmaların Analizi: Recursion Tree, Merge Sort, Recurrence Bağıntıları ve Lab-3

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Laboratuvar Sorusu Üzerinden Yol Sayma ve Kombinatorik
- 2. Özyinelemeli Bir Algoritma Nasıl Analiz Edilir?
- 3. Geriye Doğru Yerine Koyma (Backward Substitution)
- 4. Özyinelemeli Sıralı Arama: En İyi, Ortalama ve En Kötü Durum
- 5. Faktöriyelin Çağrı Ağacı ve Big-O Gösterimi
- 6. Problemi Bölerek İlerleyen Algoritmalar: Hızlı Üs Alma
- 7. İki Özyinelemeli Çağrı: Ağaç Dallanınca Ne Olur?
- 8. Merge Sort: Animasyonla Çalışması ve Analizi
- 9. Recurrence Bağıntısını Okumak: $T(n) = 2T(n/4) + n$
- 10. $T(n) = 8T(n/2) + n$
- 11. $T(n) = 8T(n/2) + n^2$
- 12. Problem, Algoritma ve Performans Karşılaştırması
- 13. Laboratuvar 3 Sorusu: Haritada En Düşük Maliyetli Yol

1. Laboratuvar Sorusu Üzerinden Yol Sayma ve Kombinatorik

Bu kayıt, üçüncü laboratuvardan sonra yapılan dersle başlar. Bu yüzden ders, laboratuvar sorusu üzerine bir konuşmayla açıldı. Soruda bir harita (kare matris) veriliyor ve iki nokta arasındaki en düşük maliyetli yol isteniyordu. Sorunun metni bu notun son bölümündedir.

Hoca önce sorunun neden “zor” sayıldığını açıkladı. Basit düşünülürse, dönem başından beri yapılan türden bir döngü çözümü vardır ve çözüm çok zor değildir. Ama problem, zorluk sınıfları açısından değerlendirildiğinde zordur. Buradaki “zor” günlük hayatta kullandığımız anlamda değildir. Hoca bununla çözümü basit bir algoritmayla kolayca yazılabilen, ama süre açısından sorun çıkaran problemleri kastetti ve bunların dönemin son dersinde biraz konuşulacağını söyledi.

1.1 Her Adımda En Küçüğü Seçmek Yeterli mi?

Soru: Gidilebilecek yolların toplamının en küçüğünü mü almalıydık, yoksa her adımda sağdaki ve aşağıdaki hücrelerden hangisi küçükse ona mı gitmeliydik? **Cevap:** Hoca iki nokta ve aralarında bir sürü yol çizdi. Yolların bazısı seni bir yönden, bazısı başka bir yönden götürüyor görünür. Esas olan yolların üzerindeki sayılardır: Örneğin bir yolda 10, 5 ve 8 vardır, başka bir yolda aradaki düğüm 18 olabilir. Matrisin üzerinden bakınca belki üstten dolaşıp gitmek uzun görünür, ama asıl seçilmesi gereken üzerindeki sayıların toplamı daha küçük olan yoldur.

1.2 Kaç Farklı Yol Var?

Hoca bu durumu lisede görülen kombinatorik sorularına benzetti: Bir noktadan başka bir noktaya hepsi eşit uzunlukta kaç farklı yol vardır? Bir öğrenci, sorudaki haritada 70 kombinasyon olduğunu, bunların hepsini toplayıp toplamayacağını ve daha kısa bir yol olup olmadığını sordu.

Hoca, bir öğrencinin yazdığı $4! \times 3!$ sonucunun nasıl çıktığını da şöyle anlattı: Sonuçta 7 adımlık bir yol gidilecektir ve adımların sırası önemlidir. İlk 4 adım $4!$ kadar, sonraki 3 adım da $3!$ kadar farklı şekilde olabilir. Buna göre sonuç $4! \times 3!$ olur.

> **Ders sonrası not:** Hoca yol sayısını $4! \times 3! = 144$ olarak hesapladı. Sağa giden adımlar kendi aralarında, aşağı giden adımlar da kendi aralarında ayırt edilemez. Bu yüzden doğru sayı $\frac{7!}{4!\,3!} = \binom{7}{3} = 35$'tir (python ile doğrulandı). Öğrencinin söylediği 70 ise 5 × 5 haritada köşeden köşeye 4 sağ ve 4 aşağı hamleli yolların sayısıdır: $\binom{8}{4} = 70$.

Hoca aynı yolları bir ağaç olarak çizmeyi de denedi. A noktasından iki yere gidilebilir. Oradan yine iki yere gidilebilir. Ağaç hep ikiye dallanarak derinleşir ve büyük, çözümü zor bir ağaç olur.

1.3 Problem ve Algoritma Farklı Şeylerdir

Soru: Bu algoritmanın adı var mı? **Cevap:** Bu bir algoritma değil, kombinatorikle ilgili bir problemdir. Hoca problem ile algoritmanın farklı kavramlar olduğunu hatırlattı: Algoritma, problemin çözümüne ilişkin bir yöntemdir. Bu konu kombinatoriğin altında sayma problemleri (counting problems) başlığında geçer, yol sayma (path counting) da bunlardan biridir.

1.4 Kombinatorik Örneği: 5 Bitle Kaç Bilgi Saklanır?

Hoca sayma problemine daha önce görülen bir örneği hatırlattı. 5 bit ile kaç farklı bilgi saklanabilir? Her bit 0 veya 1 olabilir, yani bit başına 2 seçenek vardır. Sonuç $2^5 = 32$'dir.

Bütün bilgileri kodlamak isteyen biri önce 2^n durumun hepsini belirler, sonra her duruma bir bilgi atar: `00000` A karakteri olsun, `00001` B olsun ve bu böyle gider. Bu iş 5 veya 10 bit için kolaydır, ama 100 bit için çok zorlaşır. Bu işin karmaşıklığı 2^n tarzındadır.

1.5 Polinom Olmayan Karmaşıklıklar

Kombinatorik problemlerdeki algoritmaların karmaşıklığı genelde n^2, n^3, n^4 gibi çıkmaz. Daha çok 2^n, 3^n, $n!$, n^n tarzında, polinom olmayan (non-polynomial) zamanlarla karşılaşılır. Hoca ayrıntıların son derste ve ayrık matematik, veri yapıları, algoritma analizi derslerinde farklı açılardan anlatıldığını söyledi.

Laboratuvar sorusunda olası bütün durumlara bakmak gerekiyordu. Bunun için de döngülü bir çözüm yazılıyordu. Hoca bu çözümün çok etkili olmayabileceğini, daha farklı yöntemlerin bulunduğunu söyledi. Yazılan algoritma muhtemelen faktöriyel ya da üstel bir algoritmaya doğru gidecektir. Biraz önce çizilen ağaçla yolları saymak da 2^n'lik, çok kötü bir yöntemdir.

> **Ders sonrası not:** Hoca bu problemi polinom zamanda çözülemeyen problemlere benzetti. Bütün yolları tek tek denemek gerçekten üstel sayıda yol demektir, ama problem bu yöntemle sınırlı değildir. Doğrusu büyük olasılıkla şudur: Hareket laboratuvardaki gibi hedefe doğru sınırlıysa, her hücreye en ucuz yol iki komşusundan birinden gelir. Her hücreye bir kez bakan bu çözüm (dinamik programlama) yaklaşık n^2 adımda biter. Laboratuvar haritasında (1,1)'den (4,4)'e -55 sonucunu verdiği python ile denendi. Kısa nottaki C fonksiyonu bu yöntemi gösterir. Hocanın sözünü ettiği "daha farklı yöntemler" büyük olasılıkla bunlardır.

2. Özyinelemeli Bir Algoritma Nasıl Analiz Edilir?

Hoca konuya önceki dersten hatırlatmayla girdi: Özyinelemeli algoritma, kendi kendini yineleyen algoritmadır ve en basit örneği faktöriyeldir. Bugün bunların analizine giriş yapılacaktır.

Hoca ekrandaki sözde kodların üzerinden gitti. Bu nottaki sözde kodlar, Fatih Hoca'nın aynı konudaki ders notu dosyasından (Ders 8) alındı:

```
[T]=fakto(N)
if N==1
    T=1;
else
    T=N*fakto(N-1);
end
```

Bu sözde kodda `T`, fonksiyonun döndürdüğü değeri gösterir. Hoca, MATLAB ve Python gibi dillerdeki gösterime benzediğini ve `return` ifadesinin doğrudan yazılmadığını belirtti.

2.1 Özyinelemeli Faktöriyel İyi Bir Çözüm mü?

Soru: Normal iteratif faktöriyele göre bu çözüm daha mı iyi, daha mı kötü? **Cevap:** Daha kötüdür. Adım sayısı aynıdır, yani karmaşıklık yine n'dir, ama bir de stack bellek kullanılır ve fonksiyon çağrısının getirdiği uzun işler vardır.

Hoca buna rağmen faktöriyel, Fibonacci, üs alma ve dizi elemanlarını toplama gibi örneklerin sık kullanılmasının sebebini söyledi: Bu örnekler özyinelemenin mantığını kavramak içindir. Analize de bu basit örneklerden başlanır.

2.2 Genel Yöntem: Maliyet Çarpı Çağrı Sayısı

Her özyinelemeli fonksiyonda bir temel durum (base case) bulunur. Temel durum durmayı sağlar, yani taşmayı (overflow) engeller.

Özyinelemeli algoritmanın analizi iteratif algoritmadan biraz daha zordur. İteratif algoritmada işlemleri toplam veya çarpım sembolüyle sayıp matematikteki eşitliklerle genel bir sonuca ulaşıyorduk. Burada böyle bir toplam sembolü doğrudan yoktur, çünkü problem kendini daha küçük bir parçayla tekrar eder. Bu yüzden şöyle düşünülür: Her çağrıda yapılan işlemin bir maliyeti vardır ve bu maliyet, fonksiyon kaç kez çağrılırsa o kadar tekrar eder. Kabaca:

$$\text{karmaşıklık} \approx (\text{bir çağrıdaki temel işlemin maliyeti}) \times (\text{özyinelemeli çağrı sayısı})$$

Hoca önceki derste bunun için ağaç kullanıldığını, ama basit durumlarda ağaca gerek kalmadan doğrudan koddan da yapılabileceğini söyledi.

> **Ders sonrası not:** Hoca her çağrıda kesin yapılan işlem için de "base case" dedi ("temel işlem" ile "temel durum" aynı anlamda kullanıldı). Doğrusu büyük olasılıkla şudur: Temel durum (base case) özyinelemeyi durduran durumdur, örneğin `N==1`. Her çağrıda sayılan işlem ise temel işlemdir (basic operation), örneğin `N==1` karşılaştırması. Formüldeki maliyet temel işlemin maliyetidir.

2.3 Temel İşlem Nedir?

Hoca "temel işlem" derken her hâlükârda, her çağrıda kesin kesin yapılacak işlemi kastetti. Faktöriyel kodunda bu, `N` değerinin 1 ile karşılaştırılmasıdır. Atama kesin değildir: `T=1` ataması yalnızca bazı durumlarda, n birer azalarak en sona gelindiğinde yapılır.

Bu işlemin maliyetine 1 birim denir. Hoca özellikle "bir saniye" ya da "bir milisaniye" demediğinin, "bir birim" dediğinin altını çizdi.

Peki neden 1 birim? Karşılaştırma, atama, toplama ve çıkarma assembly düzeyinde mikroişlemcide kendine ait tek bir komutla yapılır: Karşılaştırma `CMP`, toplama `ADD`, çıkarma `SUB` gibi. Bu yüzden bunlara her zaman 1 birim denebilir.

Ama temel işlemin maliyeti her zaman 1 olmak zorunda değildir. `if` içinde bir `f` fonksiyonu çağrılıyor olsaydı, `f`'in içine bakıp o işlemleri de temel işlem olarak değerlendirmek gerekirdi. Ya da çağrının içinde `if` yerine `i = 1`'den n'e kadar birer birer dönen bir `for` döngüsü olsaydı, temel işlem n kez çalışırdı. Yani çağrı başına maliyet n olurdu.

3. Geriye Doğru Yerine Koyma (Backward Substitution)

3.1 Faktöriyelde Çağrı Sayısı

Çağrı sayısını bulmak için n'den başlanır. n'i çözmek için $n-1$'e, $n-1$'i çözmek için $n-2$'ye ihtiyaç vardır ve bu böyle gider. Sonunda $n-k$'ya gelinir. $n-k$ demek, k kez çağrı yapıldı demektir. Yani bulunması gereken k'dır.

k'yı bulmak için içerideki ifade durma koşuluna eşitlenir. Durma koşulu `N==1` olduğuna göre:

$$n-k=1 \quad \Rightarrow \quad k=n-1$$

Maliyet $1 \times (n-1) = n-1$ olur. Bu tür ifadelerde sabit sayılar atılır. Yani kabaca n kez çalışır, karmaşıklık n'dir. Hoca bu işlemin genel adının geriye doğru yerine koyma (backward substitution) olduğunu söyledi: n'den $n-1$'e, $n-1$'den $n-2$'ye geriye doğru açılım yapılır ve bizi temel duruma götüren şey bulunur.

3.2 Üs Alma

```
[T]=US(x,n)
if n==1
    T=x;
else
    T=x*US(x,n-1);
end
```

Burada her çağrıda kesin yapılan iki işlem vardır: `n==1` karşılaştırması ve `x*` çarpması. Hoca bunu 2 olarak saymadı: 1, 2, 5, 10 gibi sayıların hepsi bizim için sabittir ve 1 (constant) kabul edilir.

Çağrı sayısı önceki örnekteki gibidir. `x` parametresi hiç değişmez, n ise kendisinden bir öncekini çağırır ve $n=1$'de durulur. Yaklaşık n çağrı yapılır. Karmaşıklık n'dir.

> **Hocanın vurgusu:** Ağaç da kullanılsa, doğrudan koddan analitik çözüm de yapılsa yapılacak şey aynıdır: Temel işlemin maliyetini bulmak ve bunun kaç kez tekrarlanacağını bulmak. Kaç kez tekrarlanacağını bulmak için de özyinelemeli çağrıya ve durma koşuluna bakıp bir düzen yakalanır.

3.3 $f(a,b)$ Fonksiyonu

```
[T]=f(a,b)
if b==0
    T=a;
else
    T=f(a+1,b-1);
end
```

Burada yine bir karşılaştırma vardır. Temel durum `b==0`'dır. `a` birer birer artar ama temel durumu etkilemez. Özyinelemeli çağrıyı durduracak olan `b`'dir. b, $b-1$'i çağırır, o da $b-2$'yi çağırır ve bu $b-k$'ya kadar gider. Sonunda:

$$b-k=0 \quad \Rightarrow \quad k=b$$

Fonksiyon b kez çağrılır, maliyet $1 \times b$'dir. Hoca burada b'nin sabit olmadığını, girdiye bağlı bir değer olduğunu vurguladı. Biz karmaşıklık yazarken genelde b yerine n deriz. Bunun amacı, sonucun polinom mu, üstel mi olduğunu ifade etmektir. Burada n sürekli değil, ayrık bir değişkendir ve karmaşıklık yine n'dir.

> **Ders sonrası not:** Sabit fark önemli olmadığı için hocanın sonucu doğrudur. Yalnız tam sayım yapılırsa `b==0` durumu da çağrıldığından çağrı sayısı $b+1$'dir (python ile sayıldı). Faktöriyelde ise ilk çağrıyla birlikte n çağrı yapılır.

3.4 Dizi Elemanlarını Toplama

```
[T]=f(dizi,b)
if b==1
    T=dizi(b);
else
    T=dizi(b)+f(dizi,b-1);
end
```

Hoca bunu atladı ve öğrencilere bıraktı: Dizinin eleman sayısı kadar eleman toplanacağı için karmaşıklık b, yani n çıkar. Nasıl çıktığı yukarıdaki örneklerin aynısıdır.

4. Özyinelemeli Sıralı Arama: En İyi, Ortalama ve En Kötü Durum

```
[T]=f(dizi,b,x)
if b<1
    T=-1;
else
    if dizi(b)==x
        T=b;
    else
        T=f(dizi,b-1,x);
    end
end
```

Bu fonksiyon, x değerinin dizide olup olmadığını araştırır. Aramaya dizinin sonundaki b. elemandan başlar ve geriye doğru gelir. İlk elemana kadar bulamazsa -1, yani "bulamadım" döndürür. Bulursa yerini döndürür. $b < 1$ olması da dizinin içinde bakılacak eleman kalmadığı anlamına gelir.

Burada iş biraz değişir, çünkü artık bulabilme olasılığı vardır. Hoca, C'deki gibi düşünürsek son elemanın $b-1$, ilk elemanın 0 indisinde olduğunu ve aramanın $b-1$'den 0'a doğru yapıldığını söyledi. Sonra öğrencilere bunun sonucu olarak iki olasılık olduğunu sordu. Öğrenciler önce en az bir işlem ve ilk değerde bulma gibi cevaplar verdi. Hoca bunların ileri düzey düşünmek olduğunu söyledi ve daha temelden cevabı verdi: Ya buluruz ya bulamayız.

Soru: "Hemen bulabiliriz" denebilir mi? **Cevap:** "Hemen" doğru bir ölçü değildir, çünkü herkese göre farklıdır. Ölçü işlem sayısıdır.

4.1 Bulursak ve Bulamazsak

Bulursak fonksiyon $b-1$ ile 0 arasındaki (bu değerler dahil) indislerden birinde tamamlanır ve çıkılır. Bu durum en iyi ve ortalama durumu kapsar. Bulamazsak dizinin elemanlarının hepsine bakılır, yani -1. elemana gelene kadar gidilir. Bu da en kötü durumdur.

Hoca üç durumu şöyle ayırdı:

- **En iyi durum (best case):** İlk bakılan yerde bulmak. Özyinelemeli çağrı en fazla bir kez olur.
- **Ortalama durum (average case):** Arada bir yerde, birden fazla adımda bulmak. Bu dersin konusu değildir. Hangi konumda hangi olasılıkla bulunacağı ayrıntısı sonraki derslerde görülür.
- **En kötü durum (worst case):** Dizide kaç eleman varsa o kadar, hatta bir fazlası kadar tekrar etmek.

  Ders sonrası not: Hoca ortalama durumu "arada bir yerde bulmak" olarak anlattı. Doğrusu büyük olasılıkla şudur: Ortalama durum, bütün olası girdiler üzerinden hesaplanan ortalama maliyettir. Tek bir arama için değil, olası konumların tümü için tanımlanır. En kötü durum da yalnızca "bulamamak" değildir: Bu kodda aranan eleman en son bakılan konumdaysa n çağrı, hiç yoksa $n+1$ çağrı yapılır (python ile sayıldı). Yani ikisi hemen hemen aynı maliyettedir.

4.2 En İyi ve En Kötü Durumun Maliyeti

Fonksiyonda kontrol edilen iki işlem vardır: `b<1` kontrolü ve `dizi(b)==x` kontrolü. Dolayısıyla çağrı başına 2 birimlik bir maliyet vardır.

En iyi durumda $2 \times 1 = 2$ işlem yapılır. Bunu sabit, yani 1 olarak alırız: Aranan eleman sabit zamanda bulunur.

En kötü durumda çağrı başına yine 2 işlem yapılır. Özyineleme b'yi birer azaltarak dizinin başına kadar, hatta bir de sıfırın altına düşene kadar gider. Yani yaklaşık b çağrı yapılır. Maliyet $2 \times b = 2b$'dir. Sabit atılınca karmaşıklık b, yani n olur.

Hoca burada bir genelleme yaptı: `if` veya `while` içeren durumlarda karşımıza en iyi, ortalama ve en kötü durum çıkar ve iş biraz daha karmaşıklaşır.

5. Faktöriyelin Çağrı Ağacı ve Big-O Gösterimi

Bu çözümü kağıt üzerinde elle değil de ağaçla yapmak istersek çağrı ağacı (recursion tree) kullanılır. Hoca ders notundaki ağacı gösterdi. Ağaçta her düzeyde bir çağrı vardır ve yanındaki 1 birimlik değer, o çağrıdaki temel işlemin maliyetidir. Ağaçta ayrıca bu fonksiyondan hangi fonksiyonun çağrılacağı da yazılıdır: $T(n)$, $T(n-1)$'i çağırır, o da $T(n-2)$'yi çağırır.

Ağacın yüksekliği ile her düzeydeki işlem çarpıldığında toplam işlem bulunur. Faktöriyelde yükseklik n, her düzeyin maliyeti 1'dir. Sonuç Big-O gösterimiyle $O(n)$ diye yazılır.

Hoca O dışında Big-Theta ve Big-Omega gösterimlerinin de bulunduğunu, bunların gelecek dönem ayrık matematikte veya veri yapılarında görüleceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** O büyük olasılıkla bir üst sınırı, Ω alt sınırı, Θ ise iki yönden de sıkı sınırı ifade eder. Örneğin n işlem yapan bir algoritma için $O(n)$ de $\Theta(n)$ de yazılabilir.

6. Problemi Bölerek İlerleyen Algoritmalar: Hızlı Üs Alma

Algoritma her zaman kendisinin bir eksiğini ya da iki eksiğini çağırmaz. Yarısını, üçte birini, dörtte üçünü ya da yedi bölü sekizini de çağırabilir. Hoca buna ikili aramanın (binary search) özyinelemeli yazımını örnek verdi. İkili arama yarılaya yarılaya gider. Yani n elemanlı dizi için çağrılan fonksiyon, içeride aynı diziyi artık $n/2$ elemanlıymış gibi çağırır. O da $n/4$'ü çağırır.

Bu tür fonksiyonlarda yine temel işlem bulunur, ama çağrının boyutu birer ikişer değil, yarı yarıya azalır.

6.1 Hızlı Üs Alma Kodu

```
[T]=hpower(x,p)
if p==0
    T=1;
else
    k=hpower(x,floor(p/2));
    if mod(p,2)==1
        T=x*k*k;
    else
        T=k*k;
    end
end
```

Temel işlem `p==0` karşılaştırmasıdır. Arada başka işlemler de yapılır (`mod`, çarpmalar), ama bunlar yine sabittir ve 1 alınır.

6.2 Çağrı Sayısı

Bu fonksiyon `p` üzerinden kontrol edilir. p, $p/2$'yi çağırır. $p/2$, $p/4$'ü çağırır. $p/4$, $p/8$'i çağırır. k adım sonra $p/2^k$ çağrılır.

Durma koşulu $p = 0$ olduğuna göre, öyle bir noktaya gelinir ki p 0 olur. Ama hoca burada ifadeyi 0'a eşitlemedi, 1'e eşitledi:

$$\frac{p}{2^k} = 1 \quad \Rightarrow \quad 2^k = p \quad \Rightarrow \quad k = \log_2 p$$

İki tarafın logaritması alınarak k bulunur.

Soru: Neden 1'e eşitlediniz? **Cevap:** Durma koşulu 0'dır, ama $p/2^k = 0$ dersek içler dışlar çarpımı $p = 0$ gibi bir şey verir ve k bulunamaz. Bir sayıyı böle böle tam 0 elde etmek ancak sonsuza bölmekle mümkündür. Bu yüzden 1 alınır. 1 almak çok bir kayıp yaratmaz.

Gerçekte p'den 1'e $\log_2 p$ adımda gelinir, sonra bir adımda da 0'a gelinir. Yani $k = \log_2 p + 1$'dir. Sabit olan $+1$ atılır, çünkü yanında p'ye bağlı daha büyük bir fonksiyon vardır. Temel işlemin maliyeti 1, çağrı sayısı $\log_2 p$'dir. Karmaşıklık $\log_2 n$ olur.

Bir öğrencinin sorusu üzerine hoca, tam sayı bölmesi kullanıldığı için 0'a gerçekten ulaşıldığını da belirtti: Sayılar tam sayıdır, küçülürken 1'den sonra 0 olur.

> **Ders sonrası not:** Hoca açıklama sırasında $p = 1024$ için 0'a kadar gelindiğinde "1025 tane çağrı" olduğunu söyledi. 1025, 0 ile 1024 arasındaki tam sayıların sayısıdır, çağrı sayısı değildir. `hpower` yarılayarak indiği için çağrılar 1024, 512, ..., 2, 1, 0 olur ve toplam 12 çağrı yapılır (python ile sayıldı). Yani $\log_2 p = 10$ yarılamayla 1'e, bir yarılamayla da 0'a gelinir, ilk çağrı ile birlikte 12 eder. Mertebe $\log_2 n$ olarak kalır.

Hoca sonucun önemini for döngüsüyle karşılaştırarak gösterdi: Üs almayı `for` döngüsüyle yaparken bir değişkeni n kez çarpıyorduk, yani n işlem yapıyorduk. Bu algoritmayla aynı iş $\log_2 n$ işlemde bitiyor. Daha etkili bir algoritma elde edildi.

7. İki Özyinelemeli Çağrı: Ağaç Dallanınca Ne Olur?

Hoca ders notundaki soruyu sordu: Kodda `k` değişkeni yerine aynı çağrı iki kez yazılsaydı ne olurdu?

```
T=hpower(x,floor(p/2))*hpower(x,floor(p/2))
```

Önceki kodda tek özyinelemeli çağrı vardı. Burada iki çağrı vardır.

7.1 Çağrıların Sırası

Hoca çağrılara f_1 ve f_2 diyerek ağacı $p = 8$ için çizdi:

1. $f(8)$ çalışır. Satıra gelince önce soldaki $f(4)$ çağrılır, çünkü çarpmaya geçmeden önce çağrının bitmesi gerekir.
2. $f(4)$ soldaki $f(2)$'yi, o da soldaki $f(1)$'i çağırır.
3. $f(1)$, $p = 0$ olana kadar devam ettiği için $f(0)$'ı çağırır. $f(0)$ sonucu döndürür. Ardından sağdaki $f(0)$ çağrılır. İki $f(0)$ bitince $f(1)$ biter.
4. $f(1)$ bitince $f(2)$'nin sol tarafı bitmiş olur. $f(2)$ sağ tarafındaki $f(1)$'i çağırır. O bitince $f(2)$ biter.
5. $f(2)$ bitince $f(4)$'ün sol çocuğu tamamlanır. $f(4)$ sağ taraftaki $f(2)$'yi çağırır ve aynı şeyler yeniden yapılır.
6. Sonunda $f(8)$'in soldaki $f(4)$'ü biter. $f(8)$ bir $f(4)$ daha çağırır.

> **Hocanın vurgusu:** Kod daha önce hesapladığı değeri hatırlamaz. Gidip "şuursuzca" aynı şeyleri tekrar tekrar çağırır.

Hoca önce $p = 8$ için toplamda 256 düğüm olabileceğini söyledi, sonra düzeltti: $8 = 2^3$ olduğu için yarılaya yarılaya derinlik 3 olur ve en alt düzeyde 8 tane $f(1)$ vardır.

> **Ek bilgi (derste anlatılmadı):** Bir kez hesaplanan alt problem sonucunu saklayıp yeniden kullanma fikrine büyük olasılıkla ezberleme (memoization) denir. `hpower` kodundaki `k` değişkeni de aynı alt problemi tek seferde hesaplatarak bu tekrarı önler.

7.2 Çağrı Sayısının Hesabı

Burada her çağrı kendi içinden iki alt çağrı yapar ve bunların her biri yarı boyutludur:

$$T(n) = 2T(n/2) + 1$$

Temel işlem yine bir karşılaştırmadır, yani çağrı başına 1. Bu her düzeyde böyle kalır. Çağrı sayısı ise iki katına çıkar. Hoca önce çizimde $n/4$ için $n/8$ yazıp ikişer çağrıyı atladığını fark etti ve düzeltti:

- İlk düzey: 2^1 tane $T(n/2)$
- İkinci düzey: 2^2 tane $T(n/4)$
- Üçüncü düzey: 2^3 tane $T(n/8)$
- Genel olarak: h düzey ilerlersek 2^h tane $T(n/2^h)$

Bu $n/2^h$ ifadesi 1 olduğunda dip düzeye gelinir:

$$\frac{n}{2^h} = 1 \quad \Rightarrow \quad h = \log_2 n$$

Dip düzeyde 2^h tane 1'imiz vardır. Yani bu düzeydeki düğüm sayısı $2^{\log_2 n} = n$ olur. Sonuç olarak iki kez çağıran yazımın karmaşıklığı n'dir, yani normal üs almadan farkı kalmaz.

> **Ders sonrası not:** Hoca toplamı önce "$\log_2 n \cdot 2^{\log_2 n}$" diye ifade etti. Bu çarpım $n \log_2 n$ eder, oysa ağacın düzey maliyetleri eşit değildir. Doğrusu düzeylerin toplamıdır: $\sum_{i=0}^{\log_2 n} 2^i = 2n - 1$ (ara adım eklendi, ders notunda da bu vardır). Hocanın vardığı n sonucu doğrudur. Tam sayımda p değeri 0'a kadar indiği için $p = 8$'de düzeyler 1, 2, 4, 8, 16 ve toplam 31 çağrıdır (python ile sayıldı), mertebe yine n'dir.

8. Merge Sort: Animasyonla Çalışması ve Analizi

Hoca bu algoritmayı normalde bu derste anlatmadıklarını, ama nasıl çalıştığını görmenin faydalı olacağını söyledi. Önceki derste de bahsedilmişti: Merge sort, sıralı olmayan bir diziyi sıralı hale getirir. Sıralı dizileri birleştirme yöntemini kullanır. İki sıralı diziyi birleştiren algoritma daha önce kodlanmıştı.

Soru: Sıralı olmayan bir diziyi merge ile sıralayacaksak, onu en basit kaç hale düşürmemiz lazım? **Cevap:** Bir elemana. Tek elemanlı dizi zaten sıralıdır.

8.1 Animasyondaki Çalışma

Hoca 8 elemanlı (indisleri 0'dan 7'ye) bir dizi üzerinde kodu adım adım izletti. Her çağrıda baştaki ve sondaki indise bakılır, başlangıç sondan küçükse dizinin ortası bulunur ve dizi ikiye bölünür. Sonra ilk yarı için fonksiyon yeniden çağrılır.

```
1.  (0,7) cagrilir, orta bulunur, once (0,3) cagrilir
2.  (0,3) cagrilir, orta bulunur, once (0,1) cagrilir
3.  (0,1) cagrilir, orta bulunur, once (0,0) cagrilir
4.  (0,0): baslangic sondan kucuk degil, if'in ici calismaz, geri donulur
5.  (0,1) kaldigi satirdan devam eder, (1,1) cagrilir ve hemen geri donulur
6.  Elde iki tek elemanli dizi vardir: A[0..1] birlestirilir
7.  (0,1) biter, (0,3) devam eder ve (2,3) cagrilir
8.  (2,2) ve (3,3) cagrilip geri donulur, A[2..3] birlestirilir
9.  Elde iki sirali, ikiser elemanli dizi vardir: A[0..3] birlestirilir
10. Ilk dort eleman bitmistir. Ayni islemler (4,7) icin tekrarlanir
11. En sonda A[0..3] ve A[4..7] birlestirilir
```

Hoca 4. adıma özellikle dikkat çekti: `if` koşulunun doğru olduğu durumda yapılacak bütün işlemler `if` bloğunun içindedir. Koşul sağlanmayınca hiçbiri yapılmaz ve doğrudan geri dönülür. Yani dizi en küçük parçaya bölünmüştür.

Birleştirme adımında elemanlar önce yardımcı bir diziye yedeklenir, sonra karşılaştırılıp hangisi küçükse orijinal diziye sırayla yazılır. Bu yüzden yedek bir dizi kullanmak zorunludur. Hoca önceden kodlanan birleştirme algoritmasının 2-3 farklı sürümü olduğunu, buradakinin biraz farklı olabileceğini, ama bunun önemli olmadığını söyledi.

8.2 Analizi

Bu kodda iki tür işlem vardır: bir bölme işlemi ve bir birleştirme işlemi. Bölme işlemi bir karşılaştırma ile bir orta bulmadan ibarettir ve sabittir. Birleştirmede n eleman vardır: n eleman kontrol edilir, n eleman yedek diziden yeni yerlerine kopyalanır. Yani birleştirme maliyeti n'dir.

Özyinelemeli çağrılar dizinin yarısını alır. Merge sort ve quick sort, tıpkı iki kez çağıran `hpower` gibi, iki özyinelemeli çağrı yapar. Birinci $n/2$'lik çağrı üstteki satır, ikinci $n/2$'lik çağrı alttaki satırdır. Her çağrıda da kendi boyutu kadar işlem yapılır:

$$T(n) = 2T(n/2) + n$$

Hoca düzeylerin maliyetini şöyle topladı:

- İlk düzey: n maliyet
- İkinci düzey: 2 tane $n/2$, yani $2 \cdot \frac{n}{2} = n$
- Üçüncü düzey: 4 tane $n/4$, yani $4 \cdot \frac{n}{4} = n$
- Sonrakiler: 8 tane $n/8$, 16 tane $n/16$, her biri yine n

Her düzey n olduğuna göre geriye kaç düzey olduğunu bulmak kalır. Genel olarak h. düzeyde 2^h tane $n/2^h$ vardır. Bir elemana gelene kadar bölünür, çünkü dizinin en az bir elemanı olması gerekir:

$$\frac{n}{2^h} = 1 \quad \Rightarrow \quad h = \log_2 n$$

h düzeyin her biri n maliyetli olduğundan toplam maliyet:

$$T(n) = n \cdot \log_2 n$$

Yani n^2 olan sıralama algoritmalarından çok daha iyi bir sonuca ulaşılır.

9. Recurrence Bağıntısını Okumak: $T(n) = 2T(n/4) + n$

Hoca bu kez algoritma kodu olmadan yalnızca bağıntı (recurrence relation) verilen, Fatih Hoca'nın Ders 9 notundaki sorulara geçti. Bağıntıdan ne anlaşılacağını öğrencilere sordu.

9.1 Bağıntı Ne Söylüyor?

Bir öğrenci önce her seferinde $n/4$ boyutlu bir problem çözüleceğini söyledi. Hoca bunu onayladı ve "2" sayısının ne demek olduğunu açtı: Bu kod, kendini iki kez çağırır. Bir kez $f(n/4)$ çağrılır, kodun başka bir yerinde bir kez daha $f(n/4)$ çağrılır. `hpower`'ın iki kez çağrılan yazımında olduğu gibi önce biri biter, sonra öteki başlar. Yani "$n/16$" değil, iki tane $n/4$ çağrılır. Bir sonraki düzeyde ise her $n/4$ kendi içinden iki tane $n/16$ çağırır.

Peki sondaki n nedir? Bu, ilk çağrıda yapılan işlem miktarıdır: n elemanla başlandığında temel işlem için n birimlik bir maliyet vardır. Fonksiyon kendi içinde $n/4$ için çağrıldığında bu maliyet de $n/4$ olur.

9.2 Düzey Maliyetleri

- İlk düzey: n
- İkinci düzey: 2 tane $n/4$, yani $2 \cdot \frac{n}{4} = \frac{n}{2}$
- Üçüncü düzey: 4 tane $n/16$, yani $4 \cdot \frac{n}{16} = \frac{n}{4}$

- Dördüncü düzey: 8 tane $n/64$, yani $\frac{n}{8}$

Problem küçüldükçe maliyetler de boyuta göre değişir. Hoca buradaki toplam için bir benzetme yaptı: Bir daire düşünün. Önce dairenin yarısını topladık, sonra dörtte birini, sonra sekizde birini. $\frac{n}{2} + \frac{n}{4} + \frac{n}{8} + ...$ toplamı bir tam daireye, yani n'ye doğru gider. Başındaki n ile birlikte toplam $2n$'ye yaklaşır.

9.3 Derinlik ve Toplam

Ağaç $n, n/4, n/16, n/64$ diye ilerler ve 1'e ulaşılır. h düzey sonra boyut $n/4^h$ olur:

$$\frac{n}{4^h} = 1 \quad \Rightarrow \quad h = \log_4 n$$

n parantezine alınınca toplam geometrik seri olur:

$$T(n) = n\left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + ...\right) = n \sum_{i=0}^{\log_4 n} \left(\frac{1}{2}\right)^i$$

Hocanın hatırlattığı $\sum a^i$ formülü yerine konur (ara adımlar eklendi):

$$\sum_{i=0}^{h} \left(\frac{1}{2}\right)^i = 2 - \left(\frac{1}{2}\right)^h, \qquad 2^{\log_4 n} = 2^{\frac{1}{2}\log_2 n} = \sqrt{n}$$

$$T(n) = n\left(2 - \frac{1}{\sqrt{n}}\right) = 2n - \sqrt{n}$$

Bu kapalı biçim, $T(1) = 1$ alınarak 4'ün kuvvetlerinde bağıntının kendisiyle karşılaştırıldı ve tuttu (python ile).

Soru: Peki $\sqrt{n}$ ne oldu? **Cevap:** $2n$ mi, $\sqrt{n}$ mi daha hızlı büyür? $2n$ daha hızlı büyür. Biz baskın olanı alırız. $2n$ yazılır, sabit de atılır. Karmaşıklık $O(n)$'dir.

> **Ders sonrası not:** Hoca "$2n$'in limiti $\sqrt{n}$'in limitinden daha yüksek çıkar" dedi. İkisinin de limiti sonsuzdur. Karşılaştırma oranla yapılır: $\lim_{n\to\infty} \frac{\sqrt{n}}{2n} = 0$, yani $2n$ daha hızlı büyür. Hocanın sonucu doğrudur.

Hoca burada ağaçta $n \log n$ gibi görünen bir durumdan da söz etti, ama düzey maliyetleri eşit olmadığı için karmaşıklığın yine n'ye doğru geldiğini söyledi.

> **Hocanın vurgusu:** Önemli olan, özyinelemeli çağrının kaç kez dallanacağı, temel duruma kaç adımda ulaşılacağı ve her durumda toplam ne kadar işlem yapılacağıdır. Özyinelemeli çağrılar toplam sembolünün alt ve üst sınırlarını belirler, her düzeydeki işlem de toplamın içindeki terimi verir.

10. $T(n) = 8T(n/2) + n$

Hoca bu bağıntıyı da bir öğrenciye yorumlattı. Öğrenci, kodun içinde f fonksiyonunun 8 kez çağrıldığını söyledi. Her çağrıda n, $n/2$ olur. Yani problemin uzayı yarıya iner.

Soru: Sondaki "$+n$" ne demek? **Cevap:** Hoca bir `f` fonksiyonunu kod gibi düşünmemizi istedi: İçinde bir şeyler yapılıyor, sonra f çağrılıyor, sonra bir daha çağrılıyor. "$+n$", o "bir şeyler"in maliyetidir. Bu işlemlerin çağrılardan önce olması gerekmez. Çağrıların arasında ya da sonunda da olabilir, kodların sırası bizim için önemli değildir. Önemli olan şudur: n elemanlı veri gönderildiğinde $n/2$ boyutlu çağrılar yapılır ve burada n maliyetli bir işlem yapılır. $n/2$ geldiğinde ise o işlem kendi içinde $n/2$ maliyetli olur.

10.1 Düzey Maliyetleri

- İlk düzey: 1 adet n, maliyet n
- İkinci düzey: 8 adet $n/2$, maliyet $8 \cdot \frac{n}{2} = 4n$
- Üçüncü düzey: 64 adet $n/4$, maliyet $64 \cdot \frac{n}{4} = 16n$
- Dördüncü düzey: maliyet $64n$

Kod kendi içinde 8 parçaya ayrıldığı için her düzeyde parça sayısı 8 katına çıkar: k. düzeyde 8^k düğüm vardır ve her birinin boyutu $n/2^k$'dır.

> **Ders sonrası not:** Hoca burada ağacın derinliğinin 8^k mertebesinde gittiğini söyledi. 8^k, k. düzeydeki düğüm sayısıdır. Derinlik $h = \log_2 n$'dir ve hoca birazdan bunu da bu şekilde buldu.

n parantezine alınırsa:

$$T(n) = n(1 + 4 + 16 + 64 + \dots) = n \sum_{i=0}^{h} 4^i$$

10.2 Derinlik ve Sonuç

Boyut $n/2^h$ olana kadar derinleşilir:

$$\frac{n}{2^h} = 1 \quad \Rightarrow \quad h = \log_2 n$$

Formül yerine konur:

$$T(n) = n \sum_{i=0}^{\log_2 n} 4^i = n \cdot \frac{4^{\log_2 n + 1} - 1}{4 - 1} = \frac{n}{3}\left(4 \cdot 4^{\log_2 n} - 1\right)$$

Burada logaritma işe girer. Hoca $4^{\log_2 n}$ ifadesini tahtada açtı:

$$4^{\log_2 n} = (2^2)^{\log_2 n} = 2^{2 \log_2 n} = 2^{\log_2 n^2} = n^2$$

Taban ve logaritmanın tabanı aynı olduğu için n^2 doğrudan gelir. Yerine yazılırsa:

$$T(n) = \frac{n}{3}\left(4n^2 - 1\right) = \frac{4n^3 - n}{3}$$

Bu üçüncü dereceden, yani kübik bir fonksiyondur: $O(n^3)$. Kapalı biçim, $T(1) = 1$ alınarak 2'nin kuvvetlerinde bağıntıyla karşılaştırıldı ve tuttu (python ile).

> **Ders sonrası not:** Altyazıda hoca sonucu "$4n^2$ çarpı n^3" diye söyler. Bu ifade n^5 eder ve yanlıştır. Doğrusu $4n^2 \cdot \frac{n}{3}$'tür ve n^3 mertebesini verir.

11. $T(n) = 8T(n/2) + n^2$

Son örnekte de öğrenciye önce ne okuduğu soruldu. Bir öğrenci önce n^2 kez bir şey yapıldığını söyledi. Hoca baştan başlattı: Bu algoritma kendi içinde kaç fonksiyon çağırır? 8 tane. Her biri problemi nasıl böler? $n/2$ boyutuna. İlk çağrıdaki maliyet? Doğrudan yazılıdır, n^2.

Soru: Alt çağrılar için maliyet $n/2$'nin karesi mi olur? **Cevap:** Evet, ve 8 tane olduğu için $8 \cdot (n/2)^2$ olur.

Düzeyler şöyledir:

- İlk düzey: 1 adet, maliyet n^2
- İkinci düzey: 8 adet $(n/2)^2$, yani $8 \cdot \frac{n^2}{4} = 2n^2$
- Üçüncü düzey: 64 adet $(n/4)^2$, yani $64 \cdot \frac{n^2}{16} = 4n^2$
- Dördüncü düzey: 512 adet $(n/8)^2$, yani $512 \cdot \frac{n^2}{64} = 8n^2$

n^2 parantezine alınınca toplam $n^2(1 + 2 + 4 + 8 + \dots)$ olur:

$$T(n) = n^2 \sum_{i=0}^{h} 2^i$$

Ağacın derinliği yine şöyledir: Çağrılar $n/2$ boyutuna iner ve temel durum 1'e ulaşmaktır. $n/2^h = 1$ eşitliğinden $h = \log_2 n$ bulunur.

Toplam formülü yerine konulunca (ara adım eklendi):

$$\sum_{i=0}^{\log_2 n} 2^i = 2 \cdot 2^{\log_2 n} - 1 = 2n - 1$$

$$T(n) = n^2(2n - 1) = 2n^3 - n^2$$

Hoca bu ifadeden içeriden bir n geldiğini, n^2 ile n çarpılınca yine n^3 çıktığını söyledi. Karmaşıklık $O(n^3)$'tür. Kapalı biçim, $T(1) = 1$ alınarak 2'nin kuvvetlerinde bağıntıyla karşılaştırıldı ve tuttu (python ile).

Hoca $8T(n/2) + n$ ile $8T(n/2) + n^2$ sonuçlarının aynı çıkıp çıkmadığına bir kez daha baktı. İkisinde de n^3 çıktığını, bir sorun olmadığını söyledi.

> **Ek bilgi (derste anlatılmadı):** İki bağıntıda da aynı mertebenin çıkmasının sebebi büyük olasılıkla ağacın dip düzeyidir. Dipte $8^{\log_2 n} = n^3$ tane düğüm vardır ve her düğüm yaklaşık 1 birim maliyetlidir. Üstteki düzeylerdeki n veya n^2 maliyet, bu dip düzeye göre küçük kalır.

Hoca ders notunda daha basit bir sorunun daha olduğunu söyledi ve ders notundaki kalan soruların öğrenciler tarafından çalışılmasını istedi. Ders notunda bu örneklerin ardından $T(n) = 8T(n/2) + 1$ sorusu gelir.

12. Problem, Algoritma ve Performans Karşılaştırması

Dersin sonunda hoca algoritma incelemeye dair bazı noktalara değindi.

12.1 Problem Adı Algoritma Adı Değildir

"Girilen bir sayının asal çarpanlarını bulan program" diye bir algoritma yoktur. Bu bir problemdir. "Girilen bir sayının asal çarpanlarını falanca algoritmayla bulan program" ise bir algoritmayı işaret eder. Basit bir `for` döngüsüyle yapılan iş de bu anlamda ayrı bir algoritma sayılmaz. Hoca, amaçlananın sıfırdan algoritma tasarlamak değil, tasarlanmış bir algoritmayı derste görülen ölçütlerle analiz etmek olduğunu söyledi. Bir algoritma incelendiğinde onun bir kaynaktan alınıp programa uyarlanması söz konusudur.

12.2 Zaman ve Bellek Açısından Karşılaştırma

Bir algoritma zaman ve bellek açısından farklı performans ölçütleriyle değerlendirilebilir. Hangisinin anlamlı olduğu algoritmaya göre değişir. Hoca merge sort ile quick sort'u örnek verdi: Zaman açısından ikisi de $n \log n$'dir. Ama merge sort, birleştirme sırasında ekstra bellek kullanır. Bu yüzden karşılaştırmada bellek ölçütü önem kazanır. Hoca quick sort'un ekstra bellek kullanmadığını ve bu yüzden merge sort'tan daha iyi olduğunu söyledi.

Ders sonrası not: Hoca quick sort'un $n \log n$ olduğunu ve ekstra bellek kullanmadığını söyledi. Doğrusu büyük olasılıkla şudur: $n \log n$ quick sort'un ortalama durumudur, en kötü durumda n^2'ye çıkabilir. Yardımcı dizi kullanmaz, ama özyineleme için stack belleği kullanır. Hocanın karşılaştırması, ek dizi ihtiyacı açısından geçerlidir.

12.3 Sonuçları C'de Görselleştirmek

Ölçüm sonuçları, örneğin 100 elemanda ve 500 elemanda çıkan süreler, program içinde otomatik olarak görselleştirilebilir. C'de bunun en basit yolu ölçekli bir diyagramı yıldız (*) karakterleriyle çizmektir. Histogram ya da artış-azalış grafiği böyle çizilebilir. Değerler ölçeklenir: Bir milyon tane yıldız basılmaz, değer bir milyona bölünür ve kalan sayıda yıldız basılır.

13. Laboratuvar 3 Sorusu: Haritada En Düşük Maliyetli Yol

Kaydın ikinci parçası aynı günün öğle saatinde yapılan laboratuvarın açılışıdır. Yani zamanca, kaydın başındaki dersten öncedir. Soruyu laboratuvar asistanı anlattı.

13.1 Problem

Elde bir harita (kare matris) vardır. Haritanın bazı noktaları 0'dır. Kullanıcıdan bir başlangıç ve bir bitiş noktası alınır. Bu iki nokta arasındaki en kısa yol bulunur ve yazdırılır. Her hücredeki değer mesafedir, yani o hücreye girmenin maliyetidir. Değerler negatif de olabilir. Sonda toplam yazdırılır.

Soru dosyasındaki örnek harita, indisler 0'dan başlamak üzere şöyledir:

	0	1	2	3	4
0	-10	5	30	-20	40
1	20	0	10	5	50
2	5	50	-40	15	60
3	10	-10	0	-5	70
4	15	20	30	-20	0

Asistanın saydığı istekler şunlardır:

- Harita boyutu `N`, başlangıç noktası ve hedef noktası kullanıcıdan alınır. Örnekte `N` 5'tir, ama 8 × 8 gibi başka boyutlarda da çalışmalıdır. Kod örnek haritaya göre sabit, statik yazılmamalıdır.
- Haritanın kendisi kullanıcıdan alınmaz, doğrudan koda yapıştırılır.
- Başlangıç ve bitiş noktalarının haritada gerçekten 0 olduğu kontrol edilir. Program haritayı değiştirmez, yalnızca üzerinde gezer.
- Çıktıda yolun mesafesi ve yapılan hamleler (rota) yazdırılır. Hamleler sağ, sol, yukarı ve aşağıdır, çapraz hareket yoktur. Fonksiyon ve `struct` gibi öğrenilen her şey kullanılabilir.
- Kullanıcı karanlıkta bırakılmamalıdır: Girdi beklerken boş ekran yerine her adımda açıklayıcı mesaj verilmelidir.

Soru dosyasındaki çıktıda hamleler 1 sağ, 2 sol, 3 yukarı ve 4 aşağı olarak kodlanır.

13.2 Çözülmüş İki Örnek

(1,1)'den (4,4)'e. Asistan bu yolu "sağ, aşağı, aşağı, sağ, aşağı, sağ" diye okudu. Rota `1 4 4 1 4 1` olarak yazılır. Maliyetler soru dosyasındaki çıktıyla aynıdır:

Konum	Hücre değeri	Toplam	Sonraki hamle
(1,1)	0	0	1 (sağ)
(1,2)	10	10	4 (aşağı)
(2,2)	-40	-30	4 (aşağı)
(3,2)	0	-30	1 (sağ)

Konum	Hücre değeri	Toplam	Sonraki hamle
(3,3)	-5	-35	4 (aşağı)
(4,3)	-20	-55	1 (sağ)
(4,4)	0	-55	

(3,2)'den (1,1)'e. Maliyetlerin okunuşunu asistan şöyle gösterdi: Başlangıç hücresi 0 olduğu için çıkılan hücrenin değil, girilen hücrenin değeri toplanır. Girilen hücreler -40, 10 ve 0'dır:

$$-40 + 10 + 0 = -30$$

Rota `3 3 2`, yani yukarı, yukarı, sol olur.

> **Ders sonrası not:** Altyazıda bu rota "yukarı yukarı yukarı yukarı sağ" olarak geçiyor. Soru dosyasındaki çıktıda doğrusu `3 3 2`'dir (yukarı, yukarı, sol). Asistan toplam hamleleri anlatırken de "üç sağ, dört aşağı" dedi. Birinci örneğin soru dosyasındaki çıktısı `Total Address: 1 1 1 4 4 4`'tür, yani üç sağ ve üç aşağıdır.

Asistan, hedefin doğrultusunun önemli olduğunu ekledi. (1,1)'den (4,4)'e gidilirken yalnızca sağ ve aşağı hareket edilir, geri dönülmez. Başka noktalar seçilirse hareket yönleri ona göre değişir: Başlangıç aşağıda ve hedef yukarıda olabilir, soldan sağa veya sağdan sola gidilebilir. Dört yönün dördü de geçerlidir. Asistanın örneğinde (1,2)'den (5,2)'ye gidilecekse doğrudan aşağı inilir.

13.3 Soru-Cevaplar

Soru: Değerler neyi ifade ediyor? **Cevap:** Bir hücreye gidişte katedilen mesafeyi. Yani o hamlenin maliyetini.

Soru: Girilen hücreyi mi, çıkılan hücreyi mi toplayacağız? **Cevap:** Başlangıç zaten 0 olduğu için hep girilen hücreyi almak aynı mantığı verir.

Soru: Kullanıcının girdiği başlangıç veya bitiş 0 değilse ne olacak? **Cevap:** Hata mesajı yazıp çıkılabilir, kullanıcıya yeniden de sorulabilir, ikisi de kabul edilir. Kontrol edilmezse program bir noktada hatalı çalışır.

Soru: Kullanıcı bütün sıfırları mı girecek? **Cevap:** Hayır. Haritada daha fazla 0 olsa bile kullanıcı yalnızca iki nokta girer.

Soru: Haritayı kullanıcıdan mı alacağız? **Cevap:** Hayır, bu hem sizi hem de kontrol edeni uğraştırır. Harita kodda hazır kabul edilir. Boyut için `N` alınır.

Asistan sorunun basit olduğunu, yalnızca bir noktasının kritik olduğunu, o bulunursa kısa sürede çözüleceğini söyledi. Bu noktanın ne olduğunu söylemedi.

> **Ders sonrası not:** Kritik nokta büyük olasılıkla bölüm 1.5'teki gözlemdir: Hareket hedefe doğru sınırlı olduğunda her hücreye en ucuz yol, önceki satırdaki ya da önceki sütundaki komşudan gelir. Bu yöntem yukarıdaki haritada (1,1)'den (4,4)'e -55 sonucunu verdi. Bu örnekte her adımda sağ ve aşağıdan küçüğünü seçmek de -55 veriyor (python ile denendi). Ancak bu, hocanın 1.1'de anlattığı gibi her haritada böyle değildir ve büyük olasılıkla bu haritaya özgü bir rastlantıdır. Dört yöne serbest hareket edilen genel durumda negatif döngü yoksa büyük olasılıkla Bellman-Ford gibi polinom zamanlı algoritmalar kullanılır.

Dersin Toparlaması

Bu dersin ana fikri, özyinelemeli bir algoritmanın maliyetini iki soruyla bulmaktır: Her çağrıda ne kadar iş yapılıyor ve fonksiyon kaç kez çağrılıyor? Tek özyinelemeli çağrıda bu, geriye doğru yerine koymayla bulunur: $n, n-1, \ldots, n-k$ ya da $n, n/2, \ldots, n/2^k$ yazılıp durma koşuluna eşitlenir. Faktöriyel, basit üs alma ve $f(a+1, b-1)$ bu şekilde n, hızlı üs alma ise $\log_2 n$ çıkar. Sıralı aramada ise koşullar yüzünden en iyi ve en kötü durum birbirinden ayrılır.

Bir çağrıda birden fazla özyinelemeli çağrı varsa ağaç dallanır ve düzeylerin maliyetleri toplanmalıdır. Düzeyin maliyeti, düğüm sayısı ile düğüm başına işlemin çarpımıdır. `hpower`'ın iki çağrılı yazımı n, merge sort $n \log_2 n$, $2T(n/4)+n$ bağıntısı $2n-\sqrt{n}$, $8T(n/2)+n$ ile $8T(n/2)+n^2$ ise n^3 çıkar. Hoca kalan soruların çalışılmasını istedi ve konuya ertesi günkü derste devam edileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.